

SOC-AI LOG PIPELINE

Complete Analytical Guide

Architecture · Pipeline Reference · Field Schema · Query Patterns

Powered by

LangGraph · OpenAI GPT-4.1 · Apache Kafka · OpenSearch · Fluent Bit

1. Introduction

The SOC-AI Log Pipeline is a production-grade, real-time security log analysis framework that combines Apache Kafka message streaming, Fluent Bit log collection, LangGraph AI orchestration, and OpenSearch full-text indexing into a single containerised system deployable with a single Docker Compose command.

The system solves a fundamental problem in security operations: raw syslog data from network devices is unstructured, vendor-specific, and difficult to query at scale. The pipeline automatically ingests logs from any syslog-capable device, uses GPT-4.1 to normalise them into a consistent OpenTelemetry-compatible schema, and makes them instantly queryable through a plain-English chat interface.

1.1 Design Goals

- Zero-configuration AI parsing — no regex rules or Grok patterns to maintain
- Multi-vendor support — a single system accepts logs from 18+ device types simultaneously
- Schema enforcement — every log entry is validated against a Pydantic model before storage
- Fault tolerance — failed entries are retried up to 3 times and then routed to a dead-letter queue
- Analyst-first interface — questions in plain English, structured answers with actionable insight

1.2 Technology Stack

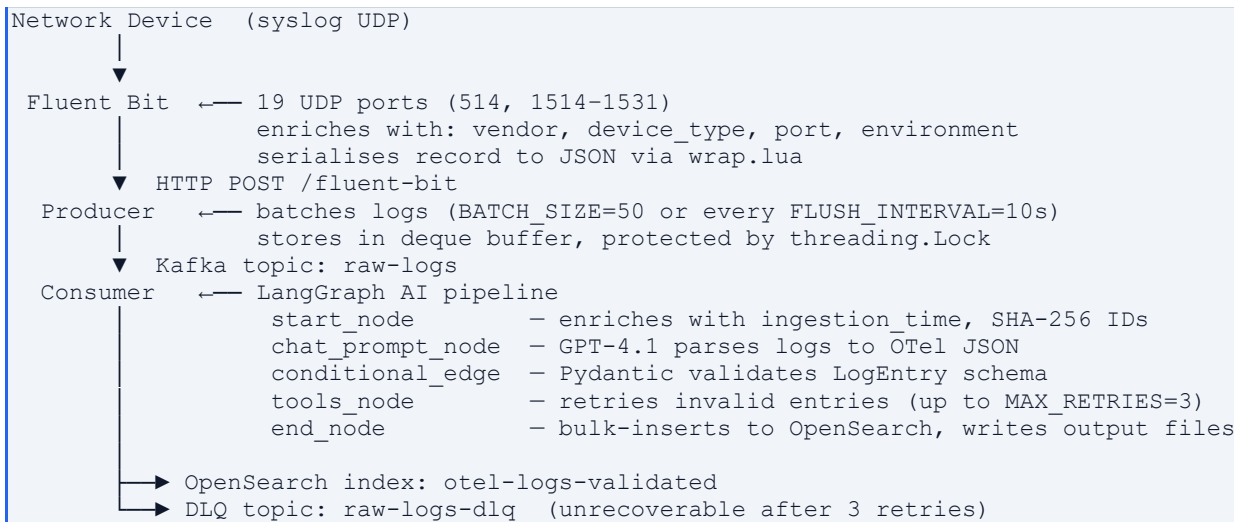
Component	Technology	Version	Role
Log Collector	Fluent Bit	Latest	Receives UDP syslog on 19 ports, enriches with vendor metadata
Message Broker	Apache Kafka (KRaft)	3.7.0	Durable buffering between producer and consumer
AI Pipeline	LangGraph + LangChain	Latest	Orchestrates multi-step LLM log parsing with retry loops
LLM	OpenAI GPT-4.1	gpt-4.1	Parses unstructured logs into structured JSON
Schema Validation	Pydantic v2	v2	Enforces LogEntry schema on every parsed entry
Log Database	OpenSearch	2.13.0	Indexes and serves validated log data
Chatbot API	FastAPI	Latest	REST API for the SOC analyst chat interface
Chatbot AI	LangGraph	Latest	Multi-node graph for intent routing and answer generation
Container Runtime	Docker Compose	v2	Single-command deployment of all 8 services

2. System Architecture

The pipeline is split into two independent sub-systems that share the OpenSearch database: the ingestion pipeline, which processes raw device logs into structured records, and the chatbot pipeline, which answers analyst questions by querying those records.

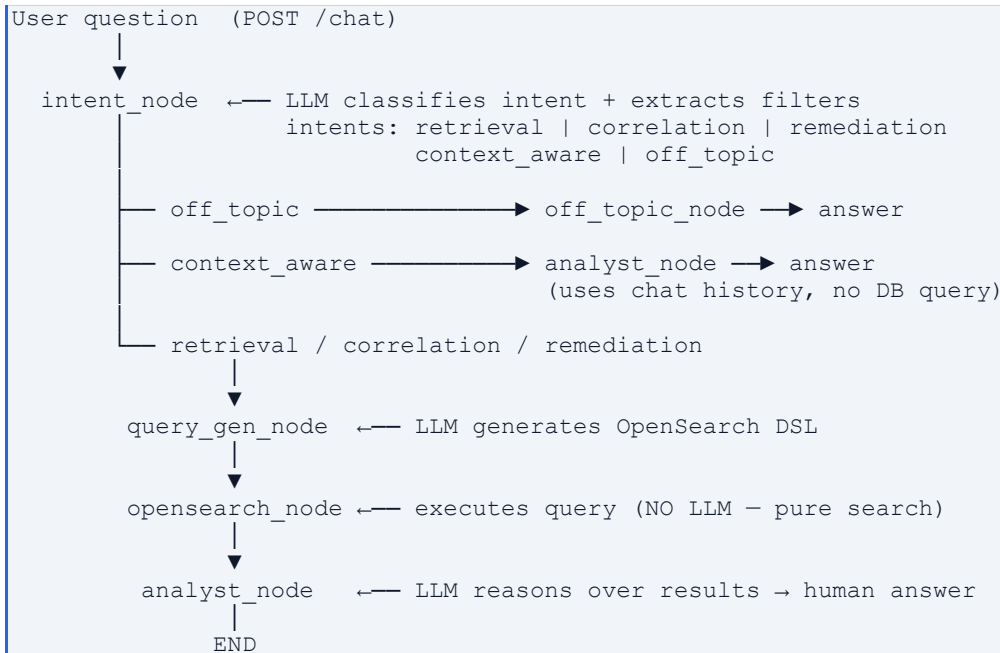
2.1 Ingestion Pipeline

Raw syslog traffic enters via Fluent Bit, flows through Kafka for durable buffering, and is processed by the LangGraph consumer before being written to OpenSearch.



2.2 Chatbot Pipeline

The SOC chatbot exposes a FastAPI REST endpoint. Each question is classified by intent, then routed to the appropriate sub-graph — either a direct answer from conversation history, or an OpenSearch query followed by AI-generated analysis.



2.3 Service Dependency Graph

Services start in dependency order enforced by Docker Compose health checks. The critical path is: Kafka → Producer → Consumer → OpenSearch → Chatbot.

Service	Port(s)	Depends on	Health check
kafka	9092 (internal) 9094 (external)	—	kafka-topics.sh --list
kafka-ui	8081	kafka (healthy)	—
producer	4318 (HTTP)	kafka (healthy)	GET /healthz
fluent-bit	514/udp, 1514–1531/udp	producer (healthy)	—
consumer	(internal)	kafka, producer, opensearch (all healthy)	—
opensearch	9200	—	curl HTTPS + grep opensearch
opensearch-dashboards	5601	opensearch (healthy)	—
chatbot	8000	opensearch (healthy)	—

3. Log Collection — Fluent Bit

Fluent Bit is the first service to receive raw device logs. It binds 19 UDP ports, one per vendor or device family, and enriches each record with vendor identification metadata before forwarding to the producer.

3.1 Port-to-Vendor Mapping

Port	Vendor / Platform	Notes
514/udp	Generic / fallback	Accepts any standard syslog format. Use this if unsure.
1514/udp	Cisco ASA, ASAv, FTD	Next-gen firewall and VPN appliances
1515/udp	Cisco IOS, IOS-XE, IOS-XR	Routers and switches
1516/udp	Cisco Firepower / FMC	Intrusion prevention and NGFW management
1517/udp	Cisco Stealthwatch	Network traffic analysis
1518/udp	Fortinet FortiGate	Enterprise next-gen firewall
1519/udp	Fortinet FortiWeb	Web application firewall (WAF)
1520/udp	Palo Alto PAN-OS (traffic)	Traffic log feed
1521/udp	Palo Alto PAN-OS (threat)	Threat/IPS log feed
1522/udp	Juniper SRX / JunOS	Enterprise firewall and routing
1523/udp	Check Point Firewall-1 / R80+	Enterprise firewall
1524/udp	Sophos XG / SFOS	UTM and NGFW
1525/udp	pfSense / OPNsense	Open-source firewall
1526/udp	WatchGuard XTM / Firebox	SMB firewall
1527/udp	Untangle NG / Arista	Network gateway and switching
1528/udp	Snort / Suricata	Open-source IDS/IPS engines
1529/udp	Zeek / Bro	Network security monitoring framework
1530/udp	IPFire / Smoothwall / iptables	Linux-based firewalls
1531/udp	Zyxel VMG / NBG / USG	Consumer and SMB routers

3.2 Record Enrichment (wrap.lua)

The Lua script `wrap.lua` serialises each Fluent Bit record into a compact JSON string and stores it in a 'log' field. This ensures the producer receives a flat, consistently shaped payload regardless of the original syslog format.

Fields added by Fluent Bit before wrap.lua executes:

- vendor — string identifying the device family (e.g. 'cisco_asa', 'fortinet_fortigate')
- device_type — category label (e.g. 'firewall', 'router', 'ids')
- port — the receiving UDP port number as a string
- environment — value of the ENVIRONMENT env var (default: 'lab')
- date — Unix epoch float timestamp assigned by Fluent Bit

3.3 HTTP Forward to Producer

Fluent Bit uses its HTTP output plugin to POST the enriched array to `http://producer:4318/fluent-bit`. The producer's `/fluent-bit` endpoint accepts both a bare JSON array and the `{'records': [...]}` envelope shape that some Fluent Bit versions produce.

4. Producer Service

The producer is a Flask HTTP server that receives log records from Fluent Bit, buffers them in an in-memory deque, and flushes batches to Kafka once `BATCH_SIZE` is reached or `FLUSH_INTERVAL` seconds have elapsed.

4.1 Batching Strategy

Batching is designed to balance throughput against latency. Two flush triggers exist simultaneously:

- Size trigger — when the buffer reaches `BATCH_SIZE` (default 50) records, a batch is immediately sent to Kafka.
- Time trigger — a background thread wakes every `FLUSH_INTERVAL` (default 10) seconds and force-flushes any partial batch, ensuring logs are never delayed more than 10 seconds.

Both triggers use a `threading.Lock` to prevent race conditions when the buffer is accessed concurrently by the HTTP request handler and the background flush thread.

4.2 HTTP Endpoints

Endpoint	Method	Purpose
/fluent-bit	POST	Receives JSON arrays from Fluent Bit. Extracts the 'log' field from each item or serialises the whole item if no 'log' key is present.
/v1/logs	POST	Receives OTLP/HTTP JSON envelopes directly. Walks the resourceLogs → scopeLogs → logRecords hierarchy and attaches resource attributes to each record.
/healthz	GET	Returns {"status": "healthy", "buffer_size": N}. Used by Docker Compose health check.

4.3 Kafka Configuration

Setting	Value	Reason
acks	all	Wait for all in-sync replicas before acknowledging — prevents data loss
retries	5	Automatically retry transient Kafka errors
linger_ms	100	Micro-batch at the network level for throughput
compression_type	gzip	Reduce network bandwidth between producer and broker

5. LangGraph Ingestion Pipeline (Consumer)

The consumer is the core AI processing component. It reads Kafka messages, runs each batch through a LangGraph state machine, validates the LLM output with Pydantic, and writes valid records to OpenSearch. Invalid records are routed to a dead-letter queue after exhausting retries.

5.1 Graph Nodes

Node	Type	Description
start_node	Deterministic	Loads the batch from state, enriches each log with an ingestion_time ISO-8601 timestamp (derived from Fluent Bit's epoch 'date' field or current UTC), and computes SHA-256 document IDs for OpenSearch deduplication.
chat_prompt_node	LLM (GPT-4.1)	Sends all pending logs as a HumanMessage to the LLM with the system prompt. Returns the raw LLM response containing a JSON array of parsed log objects.
conditional_edge_fn	Decision	Parses the LLM response, runs Pydantic validation on each entry. Returns 'end_node' if all valid or retries exhausted; returns 'tools_node' if invalid entries remain.
tools_node	Deterministic	Identifies which log indices failed validation, appends a correction request to the message history, and increments retry_count. The graph loops back to chat_prompt_node.
end_node	Mixed	Runs final Pydantic validation, bulk-inserts valid docs to OpenSearch (using SHA-256 IDs to deduplicate), appends to validated_logs.txt, routes failures to invalid_logs_buffer.txt and the DLQ Kafka topic.

5.2 Retry Logic

The pipeline retries invalid log entries up to MAX_RETRIES = 3 times. On each retry, tools_node:

1. Identifies which specific log indices failed Pydantic validation.
2. Extracts the validation error message for each failing entry.
3. Appends a new HumanMessage to the conversation asking the LLM to re-parse only the failing logs with the specific error context.
4. Loops back to chat_prompt_node.

After 3 failed retries, end_node accepts all currently valid entries and routes remaining invalid entries to the dead-letter queue and the local invalid_logs_buffer.txt file.

5.3 Pydantic Schema (LogEntry)

Every log entry must satisfy the following Pydantic model before being accepted into OpenSearch:

```
class LogEntry(BaseModel):
    timestamp: str          # ISO-8601 (from log or ingestion_time fallback)
    level: str              # ERROR | WARN | INFO | DEBUG | CRITICAL
    message: str            # Human-readable event description
    service: ServiceInfo   # name, version, env
    trace: TraceInfo        # trace_id, span_id
    event: EventInfo        # category, action, duration_ms (int)
    user: UserInfo          # id, ip
    error: Optional[ErrorInfo] # code, stack_trace (omitted if no error)
    metadata: MetadataInfo # region, container_id
```

5.4 OpenSearch Write Strategy

Valid entries are written to OpenSearch using the `helpers.bulk()` function. Each document uses the SHA-256 hash of the original raw log string as its document ID. This provides:

- Deduplication — if the same raw log is processed twice (e.g. after a consumer restart), the second write is a no-op (`raise_on_error=False` prevents crashes on version conflicts).
- Traceability — the document ID can be matched back to a raw log string for debugging.

5.5 Dead-Letter Queue (DLQ)

Unrecoverable log entries are published to the Kafka topic `raw-logs-dlq`. Each DLQ record contains:

- `raw_log` — the original raw log string as received from Kafka
- `last_llm_output` — the last dict the LLM produced for this entry
- `validation_error` — the full Pydantic validation error message
- `kafka_topic`, `kafka_partition`, `kafka_offset` — provenance for reprocessing

6. Log Schema Reference

All logs stored in OpenSearch conform to the following schema. Fields are populated by the LLM from the raw log content, with fallbacks to 'unknown' where data is absent.

6.1 Top-Level Fields

Field	Type	Example	Description
timestamp	string (ISO-8601)	2024-03-15T14:22:01Z	Event timestamp. Falls back to ingestion_time if not present in source log.
level	string	ERROR	Severity: ERROR, WARN, INFO, DEBUG, or CRITICAL
message	string	Connection refused from 10.0.0.5:443	Human-readable description of the event

6.2 Nested Object Fields

Path	Type	Example	Description
service.name	string	cisco-asa-fw01	Service or device name originating the log
service.version	string	9.14.1	Firmware or software version
service.env	string	lab	Environment tag (lab or production)
trace.trace_id	string	abc123...	Distributed trace ID (or 'unknown')
trace.span_id	string	def456...	Span ID within the trace
event.category	string	network	OpenTelemetry event category
event.action	string	connection_denied	Specific event action verb
event.duration_ms	integer	42	Event duration in milliseconds
user.id	string	admin	User or account identifier
user.ip	string	192.168.1.105	Source IP address of the actor
error.code	string	ERR_ACL_DENY	Error or fault code (optional field)
error.stack_trace	string	...	Error detail or stack trace (optional field)
metadata.region	string	eu-west-1	Geographic or logical region
metadata.container_id	string	fluent-bit	Container or node identifier

7. SOC Chatbot — Analytical Reference

The chatbot is a LangGraph-powered FastAPI service that allows security analysts to query log data in natural language. It maintains per-session conversation history, classifies question intent, generates OpenSearch DSL, and produces structured analyst-quality answers.

7.1 Intent Classification

Every question is first routed through `intent_node`, which classifies it into one of five intents and extracts structured filters. This classification determines the subsequent graph path.

Intent	When Used	Graph Path
retrieval	User wants to list or view specific log events (e.g. 'show me errors', 'list requests from Germany')	intent → query_gen → opensearch → analyst
correlation	User wants pattern analysis, anomaly detection, attack identification, or cross-service trends	intent → query_gen → opensearch (100 results) → analyst
remediation	User wants actionable fix or block steps for a security problem	intent → query_gen → opensearch → analyst
context_aware	Follow-up referencing results already shown in the conversation — no new DB query needed	intent → analyst (history only)
off_topic	Question unrelated to logs, infrastructure, or security	intent → off_topic_node → END

7.2 Filter Extraction

For retrieval, correlation, and remediation intents, the intent classifier also extracts structured filters that are injected into the OpenSearch DSL:

Filter	Type	Example	Notes
level	string	ERROR	One of: ERROR, WARN, INFO, DEBUG, CRITICAL — null if not specified
service	string	cisco-asa	Exact service name — only extracted when explicitly named in the question
keyword	string	192.168.1.5	Specific IP, username, error code, country, method. Never extracts intent words like 'suspicious' or 'errors'.
date_filter	dict	<code>{"gte": "2024-01-01", "lte": "2024-01-31"}</code>	Extracted only when a specific date or range is mentioned. Never defaults to any restriction.

7.3 Query Generation Rules

The `query_gen_node` LLM generates an initial OpenSearch DSL, but the final query is constructed by the pipeline code to prevent the LLM from injecting unintended filters:

- Structured filters (level, service, keyword) are always hard-injected using `term`, `match`, and `multi_match` clauses — the LLM cannot override or omit them.
- Date filters are only added when the user explicitly mentioned a date — the system never defaults to restricting by time.
- Correlation queries always fetch 100 results (`size=100`) regardless of LLM suggestion, to give the analyst node sufficient data for pattern detection.
- Aggregation blocks from the LLM are preserved and passed through unchanged.
- When no filters are present, the final query falls back to `match_all` so the user sees data rather than empty results.

7.4 Context-Aware Follow-ups

The chatbot maintains per-session conversation history stored in memory (`sessions` dict in `app.py`). When `context_aware` intent is detected, the `analyst_node` answers using only the existing chat history — no new OpenSearch query is executed. This allows analysts to:

- Ask 'how many of those were errors?' after a general log listing
- Request 'explain the third entry' after seeing a table
- Ask 'group those by IP' using results already in the conversation

Context-Aware Disambiguation

The intent classifier contains explicit rules to prevent misclassification of counting questions.

'How many logs are there?' → retrieval (queries OpenSearch for a count)

'How many of those logs are errors?' → `context_aware` (counts results already shown)

The rule is: if the user references prior results with pronouns ('those', 'them', 'these', 'the ones'), it is `context_aware`. Otherwise it is retrieval.

7.5 Session Management

Sessions are stored in an in-memory Python dict (`sessions: dict[str, list[dict]]`). This means sessions are lost when the container restarts. For production deployments, replace the dict with a Redis backend.

Session IDs are generated as UUIDs if not provided by the client. The client should store and re-send the `session_id` in every subsequent request to maintain conversation context.

8. REST API Reference

8.1 Chatbot API (port 8000)

POST /chat

Submit a question and receive an AI-generated answer. The endpoint runs the full LangGraph pipeline and returns a structured response.

Request body:

```
{
  "question": "Show me failed login attempts from the last hour",
  "session_id": "optional-uuid-to-continue-a-session"
}
```

Response:

```
{
  "session_id": "a1b2c3d4-...",
  "answer": "I found 14 failed login attempts in the last hour...",
  "intent": "retrieval",
  "results_count": 14
}
```

Field	Type	Description
session_id	string (UUID)	Use this in subsequent requests to maintain conversation history
answer	string	AI-generated analyst answer in Markdown format
intent	string	Classified intent: retrieval correlation remediation context_aware off_topic
results_count	integer	Number of OpenSearch documents retrieved (0 for context_aware and off_topic)

GET /history/{session_id}

Retrieve the full conversation history for a session. Returns HTTP 404 if the session does not exist.

```
{
  "session_id": "a1b2c3d4-...",
  "history": [
    { "role": "user", "content": "Show me errors" },
    { "role": "assistant", "content": "I found 3 ERROR-level events..." }
  ]
}
```

DELETE /history/{session_id}

Clear all conversation history for a session. Returns {"status": "cleared", "session_id": "..."}.

GET /health

Service liveness check. Returns {"status": "ok"} when the chatbot is running.

8.2 Producer API (port 4318)

Endpoint	Method	Description
/fluent-bit	POST	Accepts JSON arrays from Fluent Bit. Extracts 'log' field from each item.
/v1/logs	POST	Accepts OTLP/HTTP JSON envelopes (OpenTelemetry standard format).
/healthz	GET	Returns {"status": "healthy", "buffer_size": N}

9. Configuration Reference

All configuration is managed through the .env file in the project root. Values are injected into containers via docker-compose.yml environment blocks.

9.1 Environment Variables

Variable	Default	Required	Description
OPENAI_API_KEY	—	Yes	OpenAI API key. Must have access to gpt-4.1.
OPENSEARCH_PASSWORD	—	Yes	Password for the OpenSearch admin user. Used in all service-to-service connections.
OPENAI_MODEL	gpt-4.1	No	OpenAI model ID used for both log parsing and chatbot responses.
BATCH_SIZE	50	No	Number of log records per Kafka message. Larger batches improve throughput; smaller batches reduce latency.
FLUSH_INTERVAL	10	No	Seconds before a partial (< BATCH_SIZE) batch is force-flushed to Kafka.
BATCH_POLL_INTERVAL	60	No	Seconds the consumer waits between Kafka poll() calls when no messages are available.
ENVIRONMENT	lab	No	Environment tag stamped on every log record. Use 'production' for live deployments.

9.2 Tuning Recommendations

Scenario	Recommended settings
High-volume production (>1000 logs/min)	BATCH_SIZE=200, FLUSH_INTERVAL=5, BATCH_POLL_INTERVAL=10
Low-volume lab / home use (<100 logs/min)	BATCH_SIZE=20, FLUSH_INTERVAL=15, BATCH_POLL_INTERVAL=60 (defaults)
Near-real-time alerting required	BATCH_SIZE=10, FLUSH_INTERVAL=2
Cost optimisation (reduce OpenAI API calls)	BATCH_SIZE=100, FLUSH_INTERVAL=30

10. Output Files

The consumer writes two files to the outputs/ directory, which is bind-mounted from the host. Files persist across container restarts and accumulate across all Kafka messages processed since the volume was last cleared.

10.1 validated_logs.txt

Contains all successfully parsed and Pydantic-validated log entries. Each entry is a JSON object separated by a '---' delimiter. Entries are appended in the order they were processed.

```
{
  "timestamp": "2024-03-15T14:22:01Z",
  "level": "ERROR",
  "message": "Connection denied from 10.0.0.5 to 192.168.1.1:443 (ACL block)",
  "service": {"name": "cisco-asa-fw01", "version": "9.14.1", "env": "lab"},
  "trace": {"trace_id": "abc123", "span_id": "def456"},
  "event": {"category": "network", "action": "connection_denied", "duration_ms": 0},
  "user": {"id": "unknown", "ip": "10.0.0.5"},
  "metadata": {"region": "eu-west-1", "container_id": "fluent-bit"}
}
---
```

10.2 invalid_logs_buffer.txt

Contains log entries that failed Pydantic validation after all retry attempts. Each entry is a JSON object with diagnostic fields to aid in debugging or manual reprocessing.

```
{
  "raw_log": "<original raw log string>",
  "last_llm_output": {"timestamp": "unknown", "level": "INFO", ...},
  "validation_error": "1 validation error for LogEntry\n  event.duration_ms\n    value is not a valid integer",
  "batch_index": 0,
  "kafka_topic": "raw-logs",
  "kafka_partition": 0,
  "kafka_offset": 42
}
```


11. Example Query Patterns

The following table shows how different analyst questions are handled by the chatbot pipeline. Use these as a reference for understanding how the intent classifier and query generator behave.

Question	Intent	Key filters extracted	Notes
Show me the latest error logs	retrieval	level: ERROR	Returns most recent 50 ERROR entries sorted by timestamp desc
Are there any failed login attempts?	retrieval	keyword: 'failed login'	multi_match across message, event.action, error.code
Show me all traffic from IP 10.0.0.5	retrieval	keyword: '10.0.0.5'	Searches user.ip, message, and all indexed text fields
Is there anything suspicious in the last hour?	correlation	date_filter: last 1h	Fetches 100 results for pattern analysis by the analyst LLM
How do I block the IP causing the most errors?	remediation	keyword: 'errors'	Analyst LLM provides step-by-step vendor-specific block instructions
How many logs are there?	retrieval	none	match_all with track_total_hits, analyst reads the _total_hits field
What does the third entry mean?	context_aware	none	No OpenSearch query — analyst reads from chat history
How many of those were from Germany?	context_aware	none	Analyst counts from the results already in conversation history
Tell me a joke	off_topic	none	Immediately short-circuits to off_topic_node with a polite redirect

12. Monitoring Interfaces

12.1 SOC Chatbot UI (port 8000)

The primary analyst interface. A single-page web application served by FastAPI's StaticFiles middleware. The UI sends POST /chat requests and renders Markdown-formatted answers. It automatically starts a new session on first load and persists the session ID for follow-up questions.

12.2 OpenSearch Dashboards (port 5601)

A full visual log exploration interface. Login with username 'admin' and your OPENSEARCH_PASSWORD. The primary index is otel-logs-validated. From Dashboards you can:

- Build custom visualisations and dashboards
- Run raw OpenSearch DSL queries in Dev Tools
- Set up alerting rules for specific log patterns
- Explore data with the Discover interface using KQL or Lucene syntax

12.3 Kafka UI (port 8081)

A web interface for inspecting Kafka internals. Useful for diagnosing pipeline bottlenecks:

- Monitor message throughput on raw-logs and raw-logs-dlq topics
- Inspect individual message content
- Check consumer group offsets and lag
- Verify topic creation and partition counts

13. Production Considerations

13.1 Session Persistence

The current in-memory session store (dict in app.py) is lost on chatbot container restart. For production, replace it with a Redis-backed session store. The sessions dict interface is simple: get/set/delete by session_id key mapping to a list of {role, content} dicts.

13.2 OpenSearch Security

The OpenSearch instance runs with the security plugin enabled (plugins.security.disabled=false). In production:

- Use a strong, randomly generated OPENSEARCH_PASSWORD
- Enable TLS between services rather than relying on Docker network isolation
- Consider creating per-service OpenSearch users with minimum required permissions
- Enable OpenSearch audit logging to track who queried what data

13.3 Kafka Durability

The current Kafka configuration uses a single broker with replication factor 1, suitable for development. For production:s

- Deploy a 3-broker Kafka cluster for fault tolerance
- Set KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR=3 and KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR=3
- Configure log retention (log.retention.hours) to match your compliance requirements

13.4 API Key Security

The OPENAI_API_KEY is passed through Docker Compose env_file. In production:

- Use Docker secrets or a secrets manager (HashiCorp Vault, AWS Secrets Manager) instead of .env files
- Never commit the .env file to a code repository
- Rotate the API key if it is ever exposed

13.5 Cost Management

Each Kafka message batch triggers one or more GPT-4.1 API calls. LLM costs scale with log volume. To reduce costs:

- Increase BATCH_SIZE and FLUSH_INTERVAL to process fewer, larger batches
- Pre-filter trivial logs in Fluent Bit before forwarding to the producer
- Consider using a smaller model (e.g. gpt-4.1-mini) for high-volume, low-complexity logs

14. Troubleshooting

Symptom	Likely cause	Resolution
Consumer exits immediately on startup	OpenSearch not yet healthy	Wait 60-90 seconds and run docker compose up -d again; check opensearch logs with docker compose logs opensearch
Logs arrive in Kafka but not in OpenSearch	LLM returning invalid JSON	Check consumer logs: docker compose logs consumer. Review invalid_logs_buffer.txt for Pydantic error details.
Chatbot returns 'No logs found'	No data in OpenSearch yet	Verify devices are sending syslog; check Kafka UI for messages on raw-logs topic; check producer logs.
High rate of DLQ messages	Log format not matching LLM expectations	Review a sample from invalid_logs_buffer.txt; the 'last_llm_output' field shows what the LLM produced; the 'validation_error' shows which field failed.
OpenSearch Dashboards shows 'index not found'	No logs have been processed yet	Send some test syslog traffic or wait for the first batch to be consumed.
context_aware responses are wrong	LLM misclassified intent	Check the intent field in the /chat response; if retrieval was expected but context_aware was returned, rephrase the question without pronouns.